Buổi 11

Template và Nhập/xuất dòng

Học phần: Lập trình hướng đối tượng

Nội dung bài học

1. Hàm template

1.1 Vấn đề lặp mã và hàm template
1.2 Hàm template cho mảng và đa năng hóa hàm template
1.3 Template với kiểu do người dùng định nghĩa

2. Lớp template

2.1 Lớp template Stack<T>
2.2 Định nghĩa hàm thành viên ngoài lớp và tham số không phải kiểu
2.3 Lớp template Array<T> theo Quy tắc ba

3. Các dòng nhập/xuất

3.1 Dòng
3.2 Dòng xuất: <<, put, write và định dạng
3.3 Dòng nhập: >>, get, getline, read, ignore

4. Nhập/xuất file

4.1 File văn bản với ifstream và ofstream
4.2 Ghi và đọc danh sách đối tượng ra file
4.3 File nhị phân: write, read, seekg

5. Tổng kết và thực hành

5.1 Tóm tắt buổi 11
5.2 Bài thực hành buổi 11 – Stack<T> và đọc/ghi danh sách đối tượng ra file
5.3 Câu hỏi và bài tập

1.1 Vấn đề lặp mã và hàm template

- Buổi 3: muốn hoán vị int, double, char, PhanSo… phải viết **nhiều hàm đa năng hóa** với thân **giống hệt nhau**, chỉ khác kiểu ⇒ dài dòng, sửa một chỗ phải sửa tất cả.

Hàm template (hàm mẫu)

Hàm được viết với **tham số kiểu** T; trình biên dịch tự động **sinh ra** (thể hiện – instantiate) phiên bản cụ thể cho từng kiểu được dùng khi gọi. Đây là **lập trình tổng quát** (generic programming) – một dạng đa hình **lúc biên dịch**.

```
template <class T>            // hoặc template <typename T>
void HoanVi(T &a, T &b)
{  T t = a;  a = b;  b = t;  }
template <class T>
T Max(T a, T b)
{  return a > b ? a : b;  }
int main()
{  int x = 3, y = 8;          HoanVi(x, y);   // T = int
   double u = 1.5, v = 2.5;  HoanVi(u, v);   // T = double
   cout << x << " " << y << " | " << u << " " << v << endl;
   cout << Max(3, 7) << " " << Max(2.5, 1.5) << " "
        << Max('a', 'z') << " " << Max<double>(3, 4.5);
}
```

```
8 3 | 2.5 1.5
7 2.5 z 4.5
```

- **Cách hoạt động:** khi gặp `HoanVi(x, y)` với x, y kiểu int, trình biên dịch **suy ra** T = int và sinh hàm `void HoanVi(int &, int &)`. Mỗi kiểu khác nhau sinh một phiên bản riêng.
- **Suy diễn kiểu phải nhất quán:**

```
Max(3, 4.5);
// error: no matching function for call to 'Max(int, double)'
// note: deduced conflicting types for parameter 'T'
//       ('int' and 'double')
Max<double>(3, 4.5);   // chỉ định tường minh ⇒ 4.5
```

- **Yêu cầu đối với kiểu T:** hỗ trợ các phép toán dùng trong thân hàm (xem mục 1.3).

1.2 Hàm template cho mảng và đa năng hóa hàm template

```
template <class T>
void SapXep(T a[], int n)            // Bubble Sort tổng quát
{  for (int i = 0; i < n - 1; i++)
      for (int j = n - 1; j > i; j--)
         if (a[j] < a[j - 1])
            HoanVi(a[j], a[j - 1]);
}
template <class T>
void InMang(const T a[], int n)
{  for (int i = 0; i < n; i++) cout << a[i] << ' ';
   cout << endl;
}
int a[] = {5, 2, 9, 1};
SapXep(a, 4);   InMang(a, 4);        // 1 2 5 9
string s[] = {"Lan", "An", "Hoa"};
SapXep(s, 3);   InMang(s, 3);        // An Hoa Lan
```

- **Một** hàm SapXep dùng cho int, double, string, PhanSo… – miễn là kiểu có `<`, gán và `<<`.

- **Đa năng hóa hàm template:** có thể viết thêm hàm thường (hoặc template khác) cùng tên cho trường hợp đặc biệt; trình biên dịch ưu tiên **hàm thường khớp chính xác**.

```
// Max cho chuỗi C: so sánh nội dung, không so sánh địa chỉ
const char *Max(const char *a, const char *b)
{  return strcmp(a, b) > 0 ? a : b;  }
cout << Max("Hoa", "Lan");    // gọi hàm thường ⇒ Lan
```

- Nếu không có hàm thường, `Max("Hoa", "Lan")` sinh `Max<const char*>` so sánh **địa chỉ** ⇒ kết quả tùy vị trí bộ nhớ (sai).
- **Nhiều tham số kiểu:**

```
template <class T1, class T2>
void InCap(T1 a, T2 b) { cout << "(" << a << ", " << b <<
")"; }
InCap(1, "mot");   InCap(2.5, 'x');
```

So sánh	Đa năng hóa hàm	Hàm template
Số lần viết thân hàm	Mỗi kiểu một lần	Một lần
Xử lý khác nhau theo kiểu	Dễ	Cần đa năng hóa/đặc tả riêng
Kiểu chưa biết trước	Không	Có

1.3 Template với kiểu do người dùng định nghĩa

- Hàm template dùng được với **mọi kiểu** – kể cả lớp tự viết – miễn là kiểu đó cung cấp **các phép toán** mà thân hàm sử dụng (đây là "giao diện ngầm" của template).

```
class PhanSo
{  friend ostream &operator<<(ostream &os, const PhanSo &p)
   {  return os << p.Tu << "/" << p.Mau;  }
   public:
      PhanSo(int t = 0, int m = 1) : Tu(t), Mau(m) {}
      bool operator>(const PhanSo &b) const
      {  return Tu * b.Mau > b.Tu * Mau;  }
   private:
      int Tu, Mau;
};
template <class T> T Max(T a, T b) { return a > b ? a : b; }
cout << Max(PhanSo(1, 2), PhanSo(2, 3));    // 2/3
```

- **Nếu PhanSo không có `operator>`:** lỗi xuất hiện **bên trong** thân template, thông báo dài và khó đọc:

```
In instantiation of 'T Max(T, T) [with T = PhanSo]':
error: no match for 'operator>' (operand types are
       'PhanSo' and 'PhanSo')
```

- **Đọc thông báo lỗi template:** tìm dòng "with T = …" để biết kiểu nào gây lỗi, rồi bổ sung toán tử còn thiếu cho kiểu đó.
- **Đặt mã ở đâu?** Trình biên dịch phải thấy **toàn bộ thân** template tại nơi sử dụng để sinh mã ⇒ khai báo **và** định nghĩa template đặt chung trong tập tin **.h** (khác với lớp thường tách .h/.cpp).

Phép toán dùng trong template	Kiểu T phải có
`a > b`	`operator>`
`T t = a;` / `a = b;`	Copy constructor, operator=
`cout << a`	`operator<<`
`T s = 0;` / `s += x`	Constructor từ int, `operator+=`

2.1 Lớp template Stack<T> (theo giáo trình)

```
template <class T>
class Stack
{  public:
      Stack(int n = 10)
      {  Size = n > 0 ? n : 10;  Top = -1;
         StackPtr = new T[Size];  }
      ~Stack() { delete[] StackPtr; }
      bool Push(const T &x)
      {  if (IsFull()) return false;
         StackPtr[++Top] = x;  return true;  }
      bool Pop(T &x)
      {  if (IsEmpty()) return false;
         x = StackPtr[Top--];  return true;  }
      bool IsEmpty() const { return Top == -1; }
      bool IsFull() const  { return Top == Size - 1; }
   private:
      int Size, Top;
      T *StackPtr;
      Stack(const Stack &);              // cấm sao chép
      Stack &operator=(const Stack &);   // (khai báo
private)
};
```

```
Stack<int> si(5);
for (int i = 1; i <= 6; i++)
   cout << si.Push(i * 10);      // 111110
int k;
while (si.Pop(k)) cout << k << ' ';
Stack<string> ss;
ss.Push("C++");  ss.Push("hoc");  ss.Push("Toi");
string w;
while (ss.Pop(w)) cout << w << ' ';
```

```
111110
50 40 30 20 10
Toi hoc C++
```

- `Stack<int>`, `Stack<string>` là **hai lớp khác nhau** do trình biên dịch sinh ra từ cùng một mẫu.
- Lần Push thứ 6 trả về **0** vì stack đầy (kích thước 5).
- Copy constructor và operator= khai báo **private** (không định nghĩa) ⇒ cấm sao chép stack, tránh lỗi sao chép nông (buổi 8).

2.2 Định nghĩa hàm thành viên ngoài lớp và tham số không phải kiểu

- **Hàm thành viên định nghĩa ngoài lớp template** phải lặp lại `template <class T>` và ghi `Stack<T>::`:

```
template <class T>
bool Stack<T>::Push(const T &x)
{  if (IsFull()) return false;
   StackPtr[++Top] = x;
   return true;
}
```

- **Tham số template không phải kiểu** (hằng số nguyên):

```
template <class T, int N>
class Array
{  public:
      T &operator[](int i) { return A[i]; }
      int Size() const { return N; }
   private:
      T A[N];                  // kích thước cố định lúc biên
dịch
};
Array<double, 3> arr;
arr[0] = 1.5;
cout << arr.Size() << " " << arr[0];   // 3 1.5
```

- **Tham số mặc định cho template:** `template <class T = int, int N = 100> class Array;` ⇒ `Array<> a;`.
- **Lớp template và kế thừa, friend:** lớp template có thể kế thừa lớp thường hoặc lớp template khác; có thể có hàm friend template.

Ví dụ 11.1 – Thư viện chuẩn (STL) toàn là template

Lớp template	Ý nghĩa
`vector<T>`	Mảng động tự mở rộng
`list<T>`	Danh sách liên kết kép
`stack<T>`, `queue<T>`	Ngăn xếp, hàng đợi
`map<K, V>`	Từ điển (cây cân bằng)
`sort(first, last)`	Hàm template sắp xếp

```
vector<int> v = {5, 2, 9};
v.push_back(1);
sort(v.begin(), v.end());   // 1 2 5 9
```

2.3 Lớp template Array<T> theo Quy tắc ba

```
template <class T>
class Array
{  friend ostream &operator<<(ostream &os, const Array<T> &a)
   {  for (int i = 0; i < a.N; i++) os << a.A[i] << ' ';
      return os;  }
   public:
      explicit Array(int n = 0) : N(n), A(n > 0 ? new T[n] :
NULL) {}
      Array(const Array<T> &b) : N(b.N), A(b.N > 0 ? new
T[b.N] : NULL)
      {  for (int i = 0; i < N; i++) A[i] = b.A[i];  }
      ~Array() { delete[] A; }
      Array<T> &operator=(const Array<T> &b);   // như buổi 8
      T &operator[](int i);
      int Size() const { return N; }
   private:
      int N;  T *A;
};
template <class T>
T &Array<T>::operator[](int i)                  // ngoài lớp
{  if (i < 0 || i >= N) throw "Chi so ngoai mang";
   return A[i];
}
```

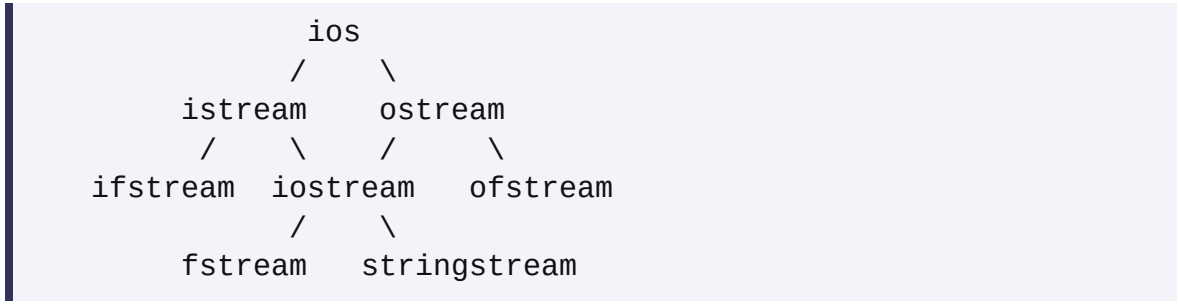
```
Array<PhanSo> ps(3);
ps[0] = PhanSo(1, 2); ps[1] = PhanSo(3, 4);
ps[2] = PhanSo(1, 3);
Array<PhanSo> b = ps;  b[0] = PhanSo(9, 9);
cout << ps << "| " << b << endl;
Array<string> s(2);  s[0] = "Xin";  s[1] = "chao";
cout << s << s.Size() << endl;
try { s[5] = "x"; }
catch (const char *loi) { cout << "Loi: " << loi << endl; }
```

```
1/2 3/4 1/3 | 9/9 3/4 1/3
Xin chao 2
Loi: Chi so ngoai mang
```

- **Một** lớp mẫu cho mảng PhanSo, string, int… với đầy đủ sao chép sâu (b sửa không ảnh hưởng ps); AddressSanitizer không báo lỗi.
- Thay `exit(1)` (buổi 8) bằng **ngoại lệ** `throw` – nơi gọi tự quyết định xử lý bằng `try … catch`.
- `new T[n]` yêu cầu T có **constructor mặc định** (PhanSo có, nhờ tham số mặc định).

3.1 Dòng (stream) và các lớp iostream

- **Dòng (stream):** chuỗi byte chảy từ nguồn tới đích. **Nhập**: byte chảy từ thiết bị (bàn phím, file, mạng) **vào** bộ nhớ; **xuất**: từ bộ nhớ **ra** thiết bị (màn hình, file).



Tập tin header	Nội dung
`<iostream>`	cin, cout, cerr, clog; istream, ostream
`<iomanip>`	setw, setprecision, setfill…
`<fstream>`	ifstream, ofstream, fstream (file)
`<sstream>`	istringstream, ostringstream (chuỗi)

Đối tượng	Lớp	Gắn với
`cin`	istream	Bàn phím (nhập chuẩn)
`cout`	ostream	Màn hình (xuất chuẩn, có bộ đệm)
`cerr`	ostream	Màn hình (lỗi, không bộ đệm)
`clog`	ostream	Màn hình (nhật ký, có bộ đệm)

- **Nhờ kế thừa:** ifstream **là một** istream ⇒ mọi thứ dùng được với cin (`>>`, getline, get…) đều dùng được với file; hàm `operator<<(ostream &, const Date &)` (buổi 8) in được ra **màn hình, file, chuỗi**.
- **Trạng thái dòng:** `good()`, `eof()` (hết dữ liệu), `fail()` (lỗi định dạng), `bad()` (lỗi nghiêm trọng); `clear()` xóa trạng thái lỗi. Dòng có thể dùng làm điều kiện: `while (cin >> x)`.

3.2 Dòng xuất: <<, put, write và định dạng

- **Toán tử chèn `<<`:** xuất mọi kiểu cơ sở, chuỗi, con trỏ (in địa chỉ), và kiểu người dùng đã đa năng hóa.
- **put(c):** xuất **một ký tự**; trả về ostream& ⇒ gọi nối tiếp.
- **write(s, n):** xuất **n byte** từ địa chỉ s (không định dạng – dùng cho dữ liệu nhị phân).

```
cout.put('O').put('K').put('\n');   // OK
cout.write("abcdef", 3);            // abc
cout << endl;
```

- **Định dạng** (buổi 2) – bằng bộ điều khiển hoặc hàm thành viên:

Bộ điều khiển	Hàm thành viên tương đương
`setw(n)`	`cout.width(n)`
`setprecision(n)`	`cout.precision(n)`
`setfill(c)`	`cout.fill(c)`
`fixed`, `scientific`	`cout.setf(ios::fixed)`
`left`, `right`	`cout.setf(ios::left, ios::adjustfield)`
`showpos`, `uppercase`	`cout.setf(ios::showpos)`

Ví dụ 11.2 – In bảng lương thẳng cột

```
cout << fixed << setprecision(0);
cout << left << setw(8) << "Ten" << right << setw(12) <<
"Luong" << endl;
cout << left << setw(8) << "An" << right << setw(12) <<
5475600 << endl;
cout << left << setw(8) << "Binh" << right << setw(12) <<
6600000 << endl;
```

```
Ten            Luong
An           5475600
Binh         6600000
```

- **Xuất ra chuỗi bằng ostringstream** – tiện để tạo chuỗi định dạng:

```
ostringstream os;
os << "SV" << setw(3) << setfill('0') << 7;
string ma = os.str();      // "SV007"
```

- **flush, endl:** `endl` = xuống dòng + đẩy bộ đệm; dùng `'\n'` khi in nhiều để chạy nhanh hơn.

3.3 Dòng nhập: >>, get, getline, read, ignore

Hàm	Hoạt động
`>> x`	Bỏ qua khoảng trắng đầu, đọc đến khoảng trắng kế tiếp
`get()`	Đọc **một** ký tự (kể cả khoảng trắng, '\n'); trả về int (EOF khi hết)
`get(c)`	Đọc một ký tự vào c; trả về istream&
`get(s, n, delim)`	Đọc tối đa n − 1 ký tự, dừng **trước** delim (delim vẫn trong dòng)
`getline(s, n, delim)`	Như get nhưng **lấy và bỏ** delim (mặc định '\n')
`getline(is, str)`	Đọc cả dòng vào `string`
`read(s, n)`	Đọc **n byte** (nhị phân); `gcount()` = số byte thực đọc
`ignore(n, delim)`	Bỏ qua tối đa n ký tự hoặc đến hết delim
`peek()`	Xem ký tự kế tiếp mà không lấy ra
`putback(c)`	Trả ký tự c lại dòng

Ví dụ 11.3 – Chạy thử trên dòng chuỗi

```
istringstream in("Nguyen Van An\nxyz 42");
char buf[30];
in.getline(buf, 30);          // đọc cả dòng 1
cout << "[" << buf << "]" << endl;
char c1 = in.get(), c2 = in.get();
cout << c1 << c2 << endl;     // từng ký tự
in.ignore(10, ' ');           // bỏ "z "
int n;  in >> n;
cout << n << endl;
```

```
[Nguyen Van An]
xy
42
```

Lỗi kinh điển: trộn >> và getline

`cin >> tuoi; getline(cin, hoTen);` ⇒ hoTen **rỗng** vì `>>` để lại '\n' trong dòng. Sửa: `cin >> tuoi; cin.ignore(1000, '\n'); getline(cin, hoTen);`.

4.1 File văn bản với ifstream và ofstream

- **Các bước:** (1) mở file – tạo đối tượng dòng gắn với tên file; (2) kiểm tra mở thành công; (3) đọc/ghi bằng `>>`, `<<`, getline… như với cin/cout; (4) đóng file – `close()` hoặc tự động trong **destructor**.

```
ofstream fo("sv.txt");              // ghi (tạo mới/ghi đè)
ofstream fa("log.txt", ios::app);   // ghi thêm vào cuối
ifstream fi("sv.txt");              // đọc
if (!fi) { cerr << "Khong mo duoc file\n"; return 1; }
```

Chế độ mở	Ý nghĩa
`ios::in`	Đọc (mặc định của ifstream)
`ios::out`	Ghi, xóa nội dung cũ (mặc định của ofstream)
`ios::app`	Ghi thêm vào cuối
`ios::binary`	Chế độ nhị phân (không chuyển đổi '\n')
`ios::in | ios::out`	Vừa đọc vừa ghi (fstream)

Ví dụ 11.4 – Đếm số dòng, số từ của file văn bản

```
ifstream f("baitap.txt");
string dong, tu;
int soDong = 0, soTu = 0;
while (getline(f, dong))            // đọc từng dòng
{  soDong++;
   istringstream ss(dong);
   while (ss >> tu) soTu++;         // tách từ
}
cout << soDong << " dong, " << soTu << " tu";
```

- Vòng `while (getline(f, dong))` dừng đúng khi hết file; **tránh** `while (!f.eof())` – vòng lặp chạy thêm một lần với dữ liệu cũ.
- **RAII:** f là đối tượng cục bộ ⇒ destructor tự đóng file khi ra khỏi phạm vi (buổi 5).

4.2 Ghi và đọc danh sách đối tượng ra file

```
class SinhVien
{  friend ostream &operator<<(ostream &os, const SinhVien &s)
   {  return os << s.Ma << ';' << s.Ten << ';' << s.DTB;  }
   friend istream &operator>>(istream &is, SinhVien &s)
   {  getline(is, s.Ma, ';');        // đọc đến dấu ;
      getline(is, s.Ten, ';');
      is >> s.DTB;
      is.ignore();                   // bỏ '\n' cuối dòng
      return is;
   }
   public:
      SinhVien(const string &m = "", const string &t = "",
               double d = 0) : Ma(m), Ten(t), DTB(d) {}
   private:
      string Ma, Ten;  double DTB;
};
int main()
{  SinhVien ds[3] = { SinhVien("SV01", "Nguyen An", 7.5),
                      SinhVien("SV02", "Tran Binh", 8.25),
                      SinhVien("SV03", "Le Chi", 6) };
   ofstream fo("sv.txt");
   for (int i = 0; i < 3; i++) fo << ds[i] << '\n';
   fo.close();
   ifstream fi("sv.txt");
   SinhVien t;  int dem = 0;
   while (fi >> t) { cout << "Doc: " << t << endl;  dem++; }
   cout << "So SV: " << dem << endl;
}
```

- **Nội dung file sv.txt:**

```
SV01;Nguyen An;7.5
SV02;Tran Binh;8.25
SV03;Le Chi;6
```

- **Kết quả chạy:**

```
Doc: SV01;Nguyen An;7.5
Doc: SV02;Tran Binh;8.25
Doc: SV03;Le Chi;6
So SV: 3
```

- **Phân tích thiết kế:**
  - Dùng dấu `;` phân cách vì họ tên có **khoảng trắng** (`>>` sẽ cắt sai).
  - `<<` và `>>` **đối xứng** (ghi thế nào đọc thế ấy) ⇒ dữ liệu đọc lại giống hệt.
  - Nhờ đa năng hóa với tham số `ostream &` / `istream &`, cùng một hàm dùng cho màn hình, bàn phím **và** file.
  - `while (fi >> t)`: `>>` trả về istream, chuyển sang bool là false khi hết file hoặc lỗi.

4.3 File nhị phân: write, read, seekg

- **File nhị phân** lưu nguyên **byte trong bộ nhớ** của dữ liệu ⇒ đọc/ghi nhanh, kích thước cố định mỗi bản ghi ⇒ **truy cập trực tiếp** bản ghi thứ k.

```
struct Diem { int x, y; };
Diem a[3] = {{1, 2}, {3, 4}, {5, 6}};
ofstream fo("diem.dat", ios::binary);
fo.write((const char *)a, sizeof(a));   // ghi 24 byte
fo.close();
ifstream fi("diem.dat", ios::binary);
fi.seekg(1 * sizeof(Diem));             // nhảy tới bản ghi 2
Diem d;
fi.read((char *)&d, sizeof(Diem));
cout << d.x << " " << d.y << " | "
     << fi.gcount() << " byte" << endl;
fi.seekg(0, ios::end);
cout << "Kich thuoc file: " << fi.tellg();
```

```
3 4 | 8 byte
Kich thuoc file: 24
```

Hàm	Ý nghĩa
`seekg(pos)` / `seekp(pos)`	Đặt vị trí đọc / ghi
`seekg(off, ios::beg/cur/end)`	Dịch so với đầu / hiện tại / cuối
`tellg()` / `tellp()`	Vị trí hiện tại

Không ghi nhị phân đối tượng có con trỏ

Ghi `write((char*)&sv, sizeof(sv))` với lớp chứa `string`, `char *` hay hàm ảo chỉ ghi **địa chỉ** (vô nghĩa khi đọc lại ở lần chạy khác). Với các lớp này hãy dùng file **văn bản** với `<<`/`>>` (mục 4.2) hoặc tự ghi từng trường.

File văn bản	File nhị phân
Đọc được bằng trình soạn thảo	Không đọc trực tiếp được
Kích thước bản ghi thay đổi	Cố định ⇒ truy cập trực tiếp
Chuyển đổi số ↔ chuỗi (chậm hơn)	Sao chép byte (nhanh)
Dễ trao đổi giữa các máy	Phụ thuộc kiến trúc máy

5.1 Tóm tắt buổi 11

Chủ đề	Ghi nhớ
Hàm template	`template <class T> T Max(T a, T b)`; trình biên dịch sinh phiên bản theo kiểu; kiểu phải nhất quán
Lớp template	`template <class T> class Stack`; dùng `Stack<int>`; hàm ngoài lớp cần `template <class T>` + `Stack<T>::`
Tham số không kiểu	`template <class T, int N>`
Dòng	istream, ostream, ifstream, ofstream, stringstream – quan hệ kế thừa
Xuất	`<<`, put, write, định dạng; ostringstream
Nhập	`>>`, get, getline, read, ignore, peek; cẩn thận trộn >> với getline
File văn bản	ofstream/ifstream; kiểm tra mở; `while (getline(f, s))`, `while (f >> x)`
File nhị phân	ios::binary, write/read, seekg/tellg – không dùng cho lớp có con trỏ

Câu hỏi nhanh

- `Stack<int>` và `Stack<double>` có dùng chung thành viên static không? → **Không** – là hai lớp khác nhau, mỗi lớp có thành viên static riêng.
- Vì sao `operator<<` nên nhận `ostream &` thay vì dùng cout? → Để dùng được cho **file** và **chuỗi** (kế thừa từ ostream).

5.2 Bài thực hành buổi 11 – Stack<T> và đọc/ghi danh sách đối tượng ra file

Bài 1 – Lớp template

- a) Cài đặt `Stack<T>` như mục 2.1 (định nghĩa hàm ngoài lớp); thêm `Top()` xem đỉnh, `Count()`.
- b) Dùng `Stack<char>` kiểm tra dấu ngoặc cân bằng; dùng `Stack<double>` tính biểu thức hậu tố "3 4 + 5 *" (đáp số **35**).
- c) Viết lớp template `Array<T>` (mảng động, Quy tắc ba, `[]` kiểm tra chỉ số, `<<`) và hàm template `SapXep`, `TimKiem`; thử với int, string, PhanSo.

Bài 2 – File

- a) Lớp `SinhVien` (Ma, HoTen, 3 điểm) với `<<` / `>>` định dạng phân cách bằng `;`.
- b) Chương trình menu: thêm SV, in danh sách, tìm theo mã, **lưu** danh sách ra `sinhvien.txt`, **nạp** lại từ file khi khởi động.
- c) Ghi báo cáo thống kê (số SV, điểm TB lớp, danh sách xếp loại) ra file `baocao.txt` với định dạng thẳng cột.
- d) (Mở rộng) Lưu file nhị phân các bản ghi kích thước cố định `struct BanGhi { char Ma[11]; char Ten[31]; float Diem; }`; sửa điểm của SV thứ k bằng `seekp` mà không ghi lại cả file.

5.3 Câu hỏi và bài tập

Câu hỏi

- 1) Hàm template khác đa năng hóa hàm ở điểm nào? Khi nào phải chỉ định kiểu tường minh `Max<double>(…)`?
- 2) Vì sao định nghĩa lớp template thường để toàn bộ trong tập tin .h?
- 3) Phân biệt `get(s, n)` và `getline(s, n)`; giải thích lỗi khi dùng getline ngay sau `cin >>`.
- 4) So sánh file văn bản và file nhị phân. Vì sao không nên ghi nhị phân đối tượng có thành viên con trỏ?

Bài tập dự đoán kết quả (có đáp số)

```
template <class T> T Tong(T a[], int n)
{  T s = 0;  for (int i = 0; i < n; i++) s += a[i];  return s;  }
int a[] = {1, 2, 3};   double b[] = {0.5, 1.5}, c[] = {1, 2};
cout << Tong(a, 3) << " " << Tong(b, 2) << " " << Tong<double>(c, 2) << endl;
istringstream in("12 abc\nxyz");  int n;  string s, t;
in >> n >> s;  getline(in, t);
cout << "|" << n << "|" << s << "|" << t << "|";
```

→ **Đáp số:** `6 2 3` (xuống dòng) `|12|abc||` – getline đọc phần còn lại của dòng 1 (chuỗi **rỗng**) vì `>>` để lại '\n'.